Visualização Geométrica, Rasterização e Recorte de Árvores Arteriais CCO

TP1 – BCC327 Computação Gráfica (2026.2)

Thiago Martins Zanete Gabriel Barony Menezes Samuel Braga Marques

Departamento de Computação (DECOM)
Universidade Federal de Ouro Preto (UFOP)

BCC327 – 2026.2

Roteiro da Apresentação

1. Contexto e Objetivos
2. Dados e Metodologia
3. Implementação
4. Resultados
5. Discussão e Conclusão

Contexto do Trabalho

Problema

Árvores arteriais geradas pelo método CCO são dados geométricos (segmentos e raios). Transformá-los em imagem envolve primitivas, transformações, rasterização e recorte, etapas que as bibliotecas gráficas costumam encapsular como uma caixa-preta.

- Modelos CCO fornecidos pelo professor em formato `.vtk`; o método CCO não foi reimplementado.
- Usamos os modelos 2D, de 8 a 256 terminais.
- Cada vaso é um segmento de reta com um atributo `raio`.
- O foco é compreender e implementar os algoritmos clássicos de Computação Gráfica.

Objetivos

Objetivo Geral

Desenvolver uma aplicação que integre leitura, visualização, rasterização e recorte de árvores CCO, deixando explícito o que foi implementado pelo grupo e o que veio de biblioteca.

- Ler o modelo `VTK` e representá-lo por linhas e tubos, com cor e espessura pelo raio.
- Aplicar translação, rotação e escala; controlar o observador; comparar projeções.
- Implementar rasterização de linhas e de polígonos e comparar com e sem antialiasing.
- Implementar o recorte de segmentos e de polígonos.

Dados Utilizados

- Arquivos: `tree2D_Nterm008`, 032, 128 e 256`.vtk`.
- Formato: VTK legado ASCII, `POLYDATA`.
- Estrutura:
  - `POINTS`: coordenadas dos pontos;
  - `LINES`: um segmento por vaso;
  - `CELL_DATA`: escalar `raio`.
- Com N terminais: $2N$ pontos e $2N - 1$ segmentos.
- Todos os modelos são planos ($z = 0$).

Modelo analisado (8 terminais)

Pontos	16
Segmentos	15
Raio mínimo	0,2490529
Raio máximo	0,6667651
Dimensão	2D

Tecnologias Utilizadas

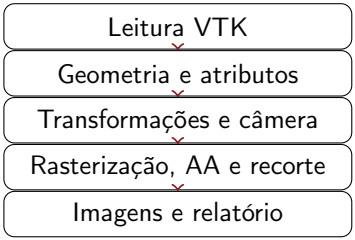
- Linguagem: Python 3.10.
- Biblioteca gráfica: PyVista/VTK, para janela, câmera, projeções e tubos.
- NumPy para as matrizes; Matplotlib apenas para exibir grades e salvar figuras.
- Entrada: arquivos `.vtk`. Saída: visualização interativa e figuras PNG.

Ambiente de execução

Ubuntu 22.04 Python 3.10 PyVista 0.49.0 NumPy 2.2.6
Intel Core i9-13900H, 16 GB Intel Arc A350M

Comando principal: `python app.py ../data/tree2D_Nterm032.vtk`

Fluxo Geral da Aplicação



Código do grupo

Leitor VTK, matrizes 4×4, Bresenham, Scanline Fill, *supersampling*, Cohen–Sutherland, Liang–Barsky e Sutherland–Hodgman.

Biblioteca (PyVista/VTK)

Janela, câmera, vistas, projeções, tubos e mapa de cores no visualizador.

Representação Geométrica

- Cada vaso é um segmento independente, com seus dois pontos próprios: um ponto compartilhado não poderia ter três raios ao mesmo tempo.
- Modos (tecla `m`): linhas coloridas pelo raio; tubos (12 lados) com espessura e cor pelo raio; tubos só com espessura.
- Raio visual reescalado: o vaso mais grosso tem 1,5% da maior dimensão do modelo.

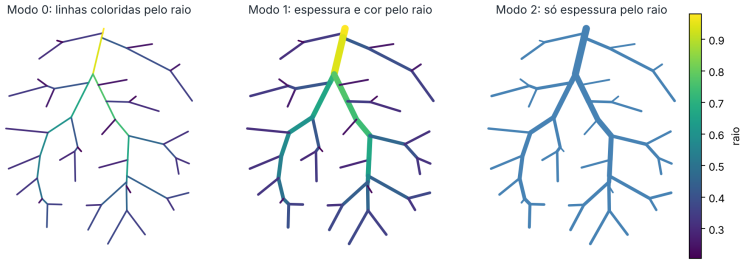
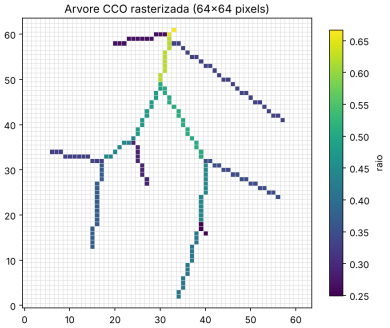


Ilustração 2D dos três modos (Matplotlib, `tree2D_Nterm032.vtk`); o app usa tubos 3D no PyVista.

Cores e Atributos Visuais

- Mapa de cores: *viridis*, contínuo, sobre $[r_{\min}, r_{\max}]$ do modelo.
- Por que contínuo: o raio é uma grandeza escalar e ordenada.
- *Viridis*: luminância monotônica, perceptualmente uniforme e legível com daltonismo.
- Barra de cores para leitura quantitativa.
- Espessura proporcional ao raio nos modos de tubo.



Árvore de 8 terminais: cor = raio

Transformações, Câmera e Projeções

- Matrizes homogêneas 4×4 (translação, escala, R_x, R_y, R_z) escritas pelo grupo.
- Rotação e escala em torno do centro c do modelo:

$$M' = T(\mathrm{c})\, R\, T(-\mathrm{c})\, M$$

- Passos: translação de 5% do tamanho, rotação de 10°, escala $\times$1,1.
- Observador: vistas `xy`, `xz`, `yz`, isométrica e mouse.

Mundo $\times$ câmera

$$\mathrm{p}_c = R_c^T(\mathrm{p}_w - \mathrm{e})$$

Projeções (tecla o)

Ortográfica: $(x_s, y_s) = (x_c, y_c)$

Perspectiva: $(x_s, y_s) = \left(\frac{f\, x_c}{z_c}, \frac{f\, y_c}{z_c}\right)$

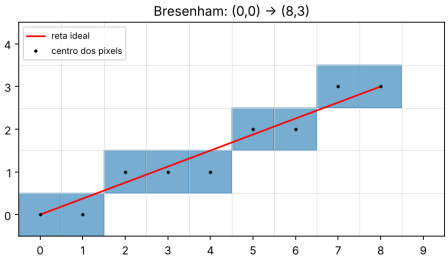
Técnicas de Computação Gráfica Implementadas

Técnica	**Como foi utilizada no projeto**
Transformações e câmera	Matrizes 4×4 próprias; vistas e projeções via PyVista
Rasterização de linhas	Bresenham (próprio), nos oito octantes
Rasterização de polígonos	Scanline Fill (próprio), regra par-ímpar
Antialiasing	*Supersampling* $k \times k$ (próprio); MSAA apenas discutido
Recorte de segmentos	Cohen–Sutherland e Liang–Barsky (próprios); Cyrus–Beck discutido
Recorte de polígonos	Sutherland–Hodgman (próprio); Weiler–Atherton discutido

Visibilidade, iluminação, transparência, textura e *ray casting* ficam para o TP2.

Rasterização de Linhas: Bresenham

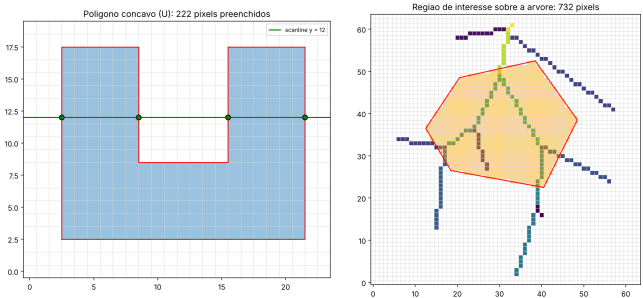
- Somente somas e comparações de inteiros.
- Erro $e = \Delta x - \Delta y$ a cada passo:
  - avança em x se $2e > -\Delta y$;
  - avança em y se $2e < \Delta x$;
  - as duas: passo diagonal.
- Gera $\max(\Delta x, \Delta y) + 1$ pixels, nos oito octantes.
- Exemplo $(0,0) \to (8,3)$: (0,0), (1,0), (2,1), (3,1), (4,1), (5,2), (6,2), (7,3), (8,3).
- Árvore de 8 terminais em 64×64: 173 pixels.



Pixels gerados e reta ideal

Rasterização de Polígonos: Scanline Fill

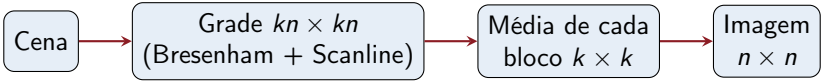
- Arestas $(y_{\min}, y_{\max}, x, \Delta x/\Delta y)$; horizontais ignoradas.
- Aresta conta se $y_{\min} \leq y < y_{\max}$ (não duplica vértices).
- Ordena os cruzamentos e pinta pares (regra par-ímpar).
- U côncavo, $y = 12$: cruzamentos 2,5; 8,5; 15,5; 21,5, pares (2,5; 8,5) e (15,5; 21,5).
- U: 222 pixels, igual à área esperada.
- Polígono raiz + folhas: 1695 pixels contra área exata de 1693,3.



Polígono côncavo e região de interesse sobre a árvore

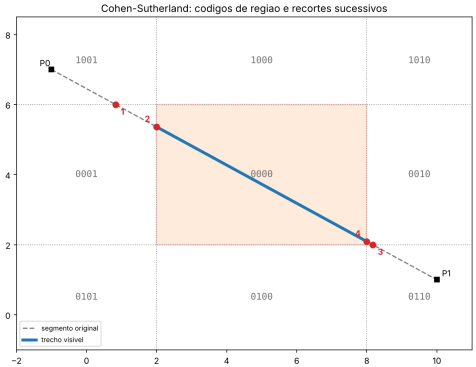
Aliasing e Antialiasing: Supersampling

- Aliasing: uma amostra por pixel em linhas inclinadas e contornos produz degraus.
- Supersampling com fator k: a cena é renderizada numa grade k vezes maior em cada eixo e cada pixel recebe a média do bloco $k \times k$ (filtro de caixa).
- Bordas ganham cobertura parcial, ou seja, tons intermediários.
- Reutiliza Bresenham (com pincel circular, para dar espessura) e Scanline Fill.
- MSAA (GPU): amostra a cobertura, mas calcula a cor uma vez por pixel. Só discutido.



Recorte de Segmentos

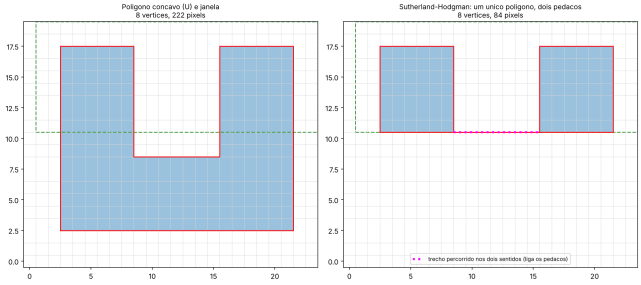
- Cohen–Sutherland: código de 4 bits por extremo. Aceita se $c_0 \mid c_1 = 0$; rejeita se $c_0 \,\&\, c_1 \neq 0$; senão move um extremo até a borda e repete.
- Liang–Barsky: $P(t) = P_0 + t\,d$ com $p_k\,t \leq q_k$ por borda. Entrada: $t_0 = \max$; saída: $t_1 = \min$; se $t_0 > t_1$, nada é visível.
- Cyrus–Beck (discutido): janela convexa qualquer, via normais. O Liang–Barsky é o caso do retângulo alinhado aos eixos.
- Exemplo: $(-1, 7) \to (10, 1)$, janela $[2, 8] \times [2, 6]$. Ambos chegam a $(2; 5{,}364) \to (8; 2{,}091)$.



Códigos de região e recortes sucessivos

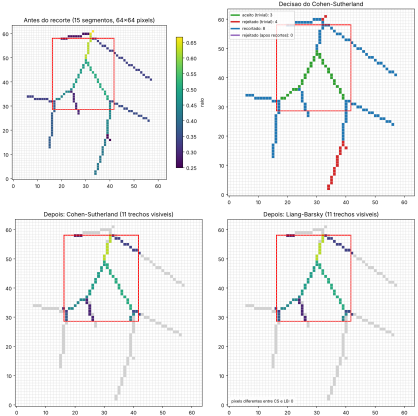
Recorte de Polígonos

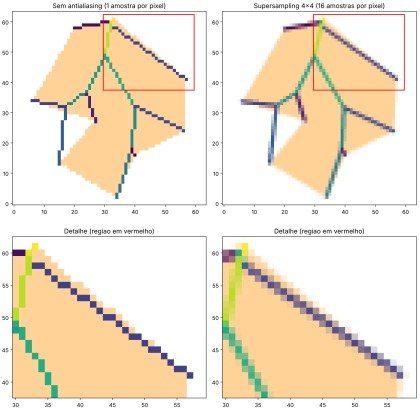
- Sutherland–Hodgman: uma borda por vez; a saída de uma é a entrada da seguinte. Quatro regras por aresta.
- Exemplo (quadrilátero): $4 \to 5 \to 6 \to 7 \to 8$ vértices, área final 86,47.
- Limitação: no U côncavo, o resultado são dois braços, mas a saída é uma lista ligada por um trecho de borda percorrido nos dois sentidos. O preenchimento ainda dá os 84 pixels corretos.
- Weiler–Atherton (discutido): um polígono por região; indicado para côncavos, várias regiões e furos.



U côncavo recortado por $y \geq 10{,}5$

Resultados Principais





Esq.: recorte da árvore em 64×64 pixels. Dos 15 segmentos, 3 foram aceitos, 4 rejeitados e 8 recortados ($173 \rightarrow 76$ pixels pintados); Cohen–Sutherland e Liang–Barsky geram a mesma imagem. **Dir.:** sem antialiasing e com *supersampling* 4×4.

Comparações Realizadas

Sem × com antialiasing (EQM)

k	1	2	4	8
EQM ($\times 10^{-3}$)	4,68	1,72	0,24	0,03

Referência: 16×16 amostras. Tons distintos: 17 (sem AA) e 200 ($k = 4$). O erro cai cerca de 20 vezes entre $k = 1$ e $k = 4$.

Cohen–Sutherland × Liang–Barsky

	CS	LB
Estratégia	códigos de região	parâmetro t
Janela	retângulo	retângulo
Resultado	idêntico	

Idênticos em 40 000 segmentos aleatórios e em 0 pixels de diferença na árvore. O custo relativo depende da implementação.

- Ortográfica × perspectiva: comparada no visualizador (tecla o) com o modelo plano inclinado.

Recurso Adicional: Recorte Interativo

Recurso implementado

Janela de recorte sobre o modelo no visualizador (`app.py`), com troca de algoritmo em tempo real.

- **Motivação:** ver o recorte atuando sobre o modelo, não só em figuras estáticas.
- **Como:** janela vermelha móvel e redimensionável (`h j k l`, `,` `.`); tecla `1` liga/desliga; tecla `2` alterna Cohen–Sutherland e Liang–Barsky; z interpolado ao longo do segmento.
- **Resultado:** árvore original em cinza, trechos visíveis coloridos pelo raio e contagem `n/N` segmentos visíveis na tela.

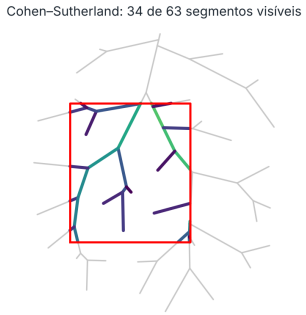


Ilustração do resultado exibido (Matplotlib, mesmo Cohen–Sutherland), 32 terminais

```
python app.py ../data/tree2D_Nterm032.vtk
-recorte -algoritmo 1
```

Discussão

- **Conceitos aplicados:** coordenadas homogêneas, mundo e câmera, projeções, Bresenham, Scanline, *supersampling*, Cohen–Sutherland, Liang–Barsky e Sutherland–Hodgman.
- **Dificuldades:** casos degenerados (vértices sobre a borda, segmentos paralelos, pontos) e convenções de pixel e de borda, resolvidos com regras explícitas e testes contra métodos independentes.
- **Com biblioteca:** janela, câmera, projeções, tubos e mapa de cores (PyVista/VTK).
- **Pelo grupo:** leitor VTK, matrizes, rasterização, antialiasing e recorte.
- **Limitações e melhorias:** só modelos 2D foram usados; o antialiasing não tem tecla no `app.py`; o recorte do app é feito no espaço do modelo; implementar Weiler–Atherton e testar modelos 3D.

Conclusões

- Os algoritmos de rasterização e recorte foram implementados e conferidos contra métodos independentes: 202 polígonos (Scanline), 40 000 segmentos (recorte) e 300 polígonos (Sutherland–Hodgman).
- O *supersampling* 4×4 reduziu o erro cerca de 20 vezes; Cohen–Sutherland e Liang–Barsky concordaram em todos os testes.
- Objetivos do TP1 cumpridos, com a divisão entre código próprio e biblioteca documentada.
- O raio como atributo, mapeado em cor e espessura, é o uso típico de visualização científica de dados vasculares.

Mensagem final

Implementar e verificar os algoritmos clássicos mostra o que o *pipeline* gráfico faz por trás da biblioteca, e é a base para visibilidade, iluminação e textura no TP2.

Referências

- SCHREINER, W.; BUXBAUM, P. Computer-optimization of vascular trees. *IEEE Transactions on Biomedical Engineering*, 1993.
- FOLEY, J. D. et al. *Computer Graphics: Principles and Practice*. Addison-Wesley.
- BRESENHAM, J. E. Algorithm for computer control of a digital plotter. *IBM Systems Journal*, 1965.
- LIANG, Y.-D.; BARSKY, B. A. A new concept and method for line clipping. *ACM Transactions on Graphics*, 1984.
- SUTHERLAND, I. E.; HODGMAN, G. W. Reentrant polygon clipping. *Communications of the ACM*, 1974.
- WEILER, K.; ATHERTON, P. Hidden surface removal using polygon area sorting. *SIGGRAPH*, 1977.
- SULLIVAN, C. B.; KASZYNSKI, A. PyVista. *Journal of Open Source Software*, 2019.
- VTK Documentation. `https://vtk.org/`

Comandos de Execução

```
cd src
python app.py ../data/tree2D_Nterm032.vtk                    #
    visualizador
python app.py ../data/tree2D_Nterm032.vtk --recorte --algoritmo 1
python bresenham.py      ../data/tree2D_Nterm008.vtk
python scanlineFill.py   ../data/tree2D_Nterm008.vtk
python antiAliasing.py   ../data/tree2D_Nterm008.vtk
python recorteSegmentos.py ../data/tree2D_Nterm008.vtk
python recorteArvore.py     ../data/tree2D_Nterm008.vtk
python recortePoligonos.py ../data/tree2D_Nterm008.vtk
```

Observação

Os módulos didáticos gravam as figuras em `../figures`.

Comandos de Teclado e Mouse (app.py)

Entrada	**Ação**
Mouse: esquerdo / roda / meio	Girar / zoom / mover a câmera
`m v o 0`	Modo de desenho; vista; ortográfica/perspectiva; restaurar o modelo
Setas; `+` `-`	Transladar; escala $\times 1{,}1$ e $\times 1/1{,}1$
`z x` / `a d` / `u i`	Rotação em Z / Y / X ($\pm 10^\circ$)
`1 2`	Liga/desliga o recorte; troca o algoritmo
`h l j k`; `,` `.`	Move a janela de recorte; reduz/amplia a janela

Verificação Automática

- **Bresenham:** extremos, contagem $\max(\Delta x, \Delta y) + 1$, pixels vizinhos e desvio $\leq 0{,}5$ da reta ideal.
- **Scanline:** comparado ao teste do raio em 202 polígonos (U, hexágono e 200 aleatórios).
- **Recorte de segmentos:** 11 casos fixos e 40 000 aleatórios; CS = LB, extremos na janela e sobre a reta, trecho maximal.
- **Sutherland–Hodgman:** 4 casos fixos e 300 polígonos aleatórios, conferidos pixel a pixel.

Núcleo do Bresenham

```
dx, dy = abs(x1 - x0), abs(y1 - y0)
sx = 1 if x0 < x1 else -1
sy = 1 if y0 < y1 else -1
erro = dx - dy
x, y = x0, y0
while True:
    pixels.append((x, y))
    if x == x1 and y == y1:
        break
    e2 = 2 * erro                    # dobro do erro evita fracoes
    if e2 > -dy:                     # precisa avancar em x
        erro -= dy
        x += sx
    if e2 < dx:                      # precisa avancar em y
        erro += dx
        y += sy
```

Obrigado!

Perguntas?

thiago.zanete@aluno.ufop.edu.br
gabriel.barony@aluno.ufop.edu.br
samuel.bm@aluno.ufop.edu.br